

DAG 任务的分区调度 考虑概率执行时间

Fuma Omori*, Atsushi Yano*[†] and Takuya Azumi*[†],

* Graduate School of Science and Engineering, Saitama University, Japan

[†] TIER IV Incorporated, Japan

摘要—自动驾驶系统与安全紧密相关, 需要实时性保证, 并可用 DAG 建模。缓存、流水线等加速机制通常使执行时间低于最坏情况。因此, 对这些系统而言, 以概率阈值约束满足条件的概率, 比最坏情况保证更合适。本文利用单处理器概率保证的研究结果, 考虑 DAG 任务的概率保证; 相较于多核处理器, 单处理器的概率保证研究已经较为成熟。本文提出一种任务集分区方法, 在分区调度下保证可调度性。在随机生成的 DAG 任务集上的评估表明, 与已有 DAG 概率可调度性分析相比, 所提方法能调度更多任务集, 且平均分析时间更短。评估还比较了四种装箱启发式, 发现以物品为中心的降序最差适配法能够调度最多的任务集。

I. 引言

自动驾驶系统 (例如 Waymo Driver [1]、Autoware [2]) 有望减少交通事故、缓解拥堵并解决物流问题。保证其实时性能, 需要确保系统处理在截止期内完成, 因此准确的响应时间分析至关重要。

自动驾驶系统的执行场景复杂, 涉及传感器数据、多个处理过程及其相互依赖, 因而难以建模。为此采用有向无环图 (DAG) [3]–[5]: 以节点表示系统处理过程, 以有向边表示依赖, 从而简化响应时间分析。

硬实时 DAG 任务系统的现有研究, 基于每个节点的最坏情况执行时间 (WCET) 分析响应时间 [3], [6]。然而, 现代自动驾驶硬件中的缓存、流水线等机制, 经常使任务执行明显更快; 基于 WCET 得到的响应时间因而过于悲观, 妨碍实际系统设计。实时调度中的概率方法解决这一问题: 根据实际统计数据, 把节点执行时间建模为概率分布, 并提供满足截止期的概率保证。

概率最坏情况执行时间 (pWCET) [7] 从概率角度对执行时间作安全的上界近似。它之所以常用, 是因为允许采用独立同分布 (IID) 的概率假设: 尽管缓存等因素会造成依赖, 仍可将各处理过程的执行时间独立处理。pWCET 分布通过静态概率时序分析 (SPTA) 和基于测量的概率时序分析 (MBPTA) 获得 [8]–[10]。本文假设 pWCET 分布已经给定, 不研究其获取方法。

对于具有 pWCET 或概率执行时间的 DAG 任务, 目前只有少数概率响应时间方法 [11]–[13]。一般而言, 概率分析需要考虑多个状态, 因此这些分析方法只能处理较小的任务集。与 DAG 任务分析相比, 单核概率分析 [14]–[18] 更为成熟, 并已有缩短分析时间的技术。

本文提出一种分区最早截止期优先 (EDF) 调度下的 DAG 任务集分区方法。DAG 任务的每个节点均具有概率执行时间。

本文的贡献如下:

本研究获得日本 JST AIP Acceleration Research 项目 JP-MJCR25U1 和 JST CREST 项目 (编号 JPMJCR23M1) 支持。

表 I: 记号汇总

符号	含义
$\Gamma = \{\tau_1, \dots, \tau_n\}$	待调度的任务集
$\tau_i = (G_i, T_i, D_i, \rho_i)$	DAG 任务
$G_i = (V_i, E_i)$	DAG
T_i	τ_i 的周期
D_i	τ_i 的相对截止期
ρ_i	指定 τ_i 最大可接受截止期违约概率的阈值
V_i	DAG G_i 的节点集合
E_i	DAG G_i 的边集合
$v_{i,j} \in V_i (j \in \{1, \dots, V_i \})$	DAG G_i 的节点 (子任务)
$(v_{i,j}, v_{i,k}) \in E_i$	DAG G_i 中从节点 $v_{i,j}$ 到 $v_{i,k}$ 的边
$C_{i,j}$	节点 $v_{i,j}$ 的 pWCET 分布
C_i	定义 1 定义的概率总执行时间分布
$J_{i,j}$	$\tau_i = (C_i, T_i, D_i, \rho_i)$ 释放的第 j 个作业
$d_{i,j}$	$J_{i,j}$ 的绝对截止期
C'_i	定义 2 定义的调整后概率总执行时间分布
U'_i	定义 3 定义的 τ_i 调整后平均利用率
U'_A	定义 4 定义的任意任务集 A 的调整后平均利用率
Φ	截止期违约概率
S_I	释放时间与截止期均位于区间 I 内的作业执行时间之和

- 考虑概率执行时间后, 所提分区方法能够调度最坏情况利用率超过一的任务集, 而传统的 WCET 方法不能。这提高了可调度性, 减少悲观性, 并改善系统效率。
- 在 EDF 下将 DAG 任务视为单节点任务, 使所提方法在可调度任务集数量和平均分析时间方面优于已有可调度性分析, 并能够分析此前难以处理的任务集。
- 在任务集分区时采用以物品为中心的降序最差适配策略, 与其他装箱策略相比, 能使所提方法在利用率更高的系统中保证可调度性。

本文余下内容安排如下。第 II 节描述系统模型; 第 III 节介绍所提方法; 第 IV 节说明如何将所提方法应用于联邦调度; 第 V 节评估并讨论所提方法; 第 VI 节介绍相关工作; 第 VII 节给出结论和未来工作。

II. 系统模型

本节描述系统模型。本文记号列于表 I。

假设系统由多个同构处理器核和 DAG 任务集 $\Gamma = \{\tau_1, \dots, \tau_n\}$ 组成。每个 DAG 任务 τ_i 定义为 (G_i, T_i, D_i, ρ_i) , 其中 $G_i = (V_i, E_i)$ 是由节点集合与边集合构成的 DAG。 T_i 和 D_i 为正整数, 分别表示 DAG 任务的周期与相对截止期。 T_i 与 D_i 满足 $D_i \leq T_i$, 即采用约束截止期。

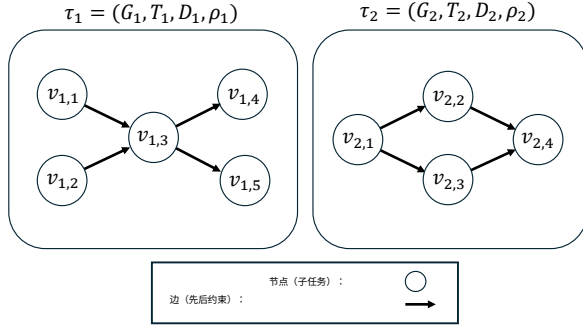


图 1: DAG 任务模型示例：同一框内的子任务具有相同的相对截止期与周期。

如果估计的截止期违约概率低于阈值 ρ_i ，则认为 τ_i 可调度 [18]。类似地，如果任务集中每个估计截止期违约概率均低于对应阈值，则认为该任务集可调度。

DAG 的每个节点 $v_{i,j} \in V_i$ ($j \in \{1, \dots, |V_i|\}$) 都是一个子任务，由 pWCET 分布 $C_{i,j}$ 描述。pWCET 分布 $C_{i,j}$ 是离散概率分布，其中 $C(x)$ 为执行时间等于 x 的概率。服从分布 C 的随机变量记为 $\mathcal{C}$ 。DAG 的每个子任务直接继承 DAG 任务的周期和相对截止期；各子任务的作业以偶发方式同步释放，相邻释放间隔至少为一个周期。假设超过截止期的作业将被终止，并允许在任意时刻抢占。

边 $(v_{i,j}, v_{i,k}) \in E_i$ 表示：子任务 $v_{i,j}$ 的第 ℓ 个作业必须完成后，子任务 $v_{i,k}$ 的第 ℓ 个作业才可以开始。系统模型示例见 图 1。两个圆角框分别表示两个 DAG 任务。

每个 DAG 任务分配到一个处理器核，并在该核上按 EDF 调度。在这一调度假设下，每个 DAG 任务 τ_i 可表示为 (C_i, T_i, D_i, ρ_i) ，其中 C_i 定义如下：

定义 1 (概率总执行时间分布). 任务 τ_i 的概率总执行时间分布 C_i ，由各节点的 pWCET 分布和算子 $\otimes$ 定义为

$$C_i = \bigotimes_{j=1}^{|V_i|} C_{i,j}. \quad (1)$$

算子 $\otimes$ 表示卷积：对于分布 X 与 Y ，有 $(X \otimes Y)(x) = \sum_{k=-\infty}^{\infty} \mathbb{P}(X = k) \mathbb{P}(Y = x - k)$ 对 $x \in \mathbb{R}$ 。

任务 τ_i 可表示为 (C_i, T_i, D_i, ρ_i) ，因为按照 G_i 的拓扑顺序依次调度节点，可以遵守先后约束 E_i 。另外，概率总执行时间 C_i 与 V_i 中节点的执行顺序无关，因为每个节点 $v_{i,j} \in V_i$ 的概率执行时间，服从独立的执行时间分布 $C_{i,j}$ 。将 τ_i 的第 j 个作业记为 $J_{i,j}$ ，其绝对截止期为 $d_{i,j}$ 。

基于概率总执行时间分布（定义 1），对每个任务作如下假设。

假设. 对 Γ 中的每个任务 τ_i ，以下不等式成立。

$$\mathbb{P}(C_i > D_i) \leq \rho_i \quad (2)$$

C_i 是服从 C_i 的随机变量。

该假设意味着：如果没有其他任务调度到 τ_i 所在的处理器核，则每个任务 τ_i 都可调度。

III. 所提出的分区算法

本节详细说明所提方法：在分区 EDF 下，计算处理器核数量并对 DAG 任务集 Γ 分区。内容安排为：面向概率分区的利用率相关定义（第 III-A 节）、所提分区算法（第 III-B 节）、采用的可调度性分析（第 III-C 节），以及过载概率的计算（第 III-D 节）。

A. 预备定义

本节定义所提方法使用的调整后平均利用率。为此，先定义调整后概率总执行时间分布：

定义 2 (调整后概率总执行时间分布). 任务 τ_i 的调整后概率总执行时间分布 C'_i 定义为

$$C'_i(c) = \mathbb{P}(C'_i = c) = \begin{cases} \mathbb{P}(C_i = c) & \text{若 } c < D_i \\ \sum_{k=c}^{\infty} \mathbb{P}(C_i = k) & \text{若 } c = D_i \\ 0 & \text{若 } c > D_i \end{cases} \quad (3)$$

$\forall c \in \text{Im } C_i$ 均为一个执行时间。

利用调整后概率总执行时间分布，定义任务与任务集的调整后平均利用率如下：

定义 3 (任务的调整后平均利用率). 设 C'_i 为 τ_i 的调整后概率总执行时间分布。 τ_i 的调整后平均利用率 $\bar{U}'_i$ 定义为

$$\bar{U}'_i = \frac{\mathbb{E}[C'_i]}{T_i}. \quad (4)$$

定义 4 (任务集的调整后平均利用率). 对于任意任务集 A ，其调整后平均利用率定义为

$$\bar{U}'_A = \sum_{\tau_i \in A} \bar{U}'_i. \quad (5)$$

由定义 2，每个任务 $\tau_i \in \Gamma$ 的调整后平均利用率 $\bar{U}'_i$ 大于零且小于或等于一。

调整后平均利用率考虑了截止期终止策略，忽略已经超过截止期的作业。普通平均利用率在不采用定义 2 时可能超过一（例如， $T_i = 5, C_i = \begin{pmatrix} 1 & 10^3 \\ 1 - 10^{-2} & 10^{-2} \end{pmatrix}$ 则 $\bar{U} = \mathbb{E}[C_i]/T_i = 2.198 > 1$ ），而调整后的版本始终位于 $(0, 1]$ ，从而便于任务集分区。

B. 分区

本小节详细介绍所提分区过程，目标是在保证 Γ 可调度的同时，最小化所需处理器核数量。例如，如果三个任务 τ_1, τ_2, τ_3 分别可调度（依据第 II 节中的**假设**），最初需要三个核。如果 τ_1 与 τ_2 可以共同调度，核数需求就减少到两个。

虽然遍历所有分区方式可以最小化核数，但分区数量按照 Bell 数增长，只能用于较小的任务集。因此，实际应用需要启发式，以较少分区进行高效划分。

所提分区方法借鉴四种启发式的思路：以物品为中心的降序最佳适配（ICBFD）、以物品为中心的降序最差适配（ICWFD）、以箱子为中心的最佳适配（BCBF），以及以箱子为中心的最差适配（BCWF）。这四种方法已用于装箱问题的启发式 [19], [20]。前缀 IC（以物品为中心）和 BC（以箱子为中心）表示两类传统装箱算法。IC 逐个考虑物

算法 1: 用 ICBFD 寻找分区

输入: Γ
输出: Γ 的一个分区

```
1  $sorted\_partition \leftarrow \{\};$  // 用于保存各任务列表
   的列表
2  $sorted\_free\_tasks \leftarrow$ 
    $sort\_by\_adjusted\_utilization(\Gamma);$  // 按降序
   排列
   //  $sorted\_free\_tasks$  的首部
3 对于  $task \in sorted\_free\_tasks$  执行
4    $core\_id \leftarrow$ 
      $find\_core\_id(sorted\_partition, task);$ 
5   若  $core\_id$  为  $None$  则
6     将  $\{task\}$  加入  $sorted\_partition;$ 
7   否则
8     将  $task$  加入  $sorted\_partition[core\_id];$ 
9     若 不满足
        $is\_feasible(sorted\_partition[core\_id])$  则
10       从  $sorted\_partition[core\_id]$  移除  $task;$ 
11       将  $\{task\}$  加入  $sorted\_partition;$ 
12     结束
13   结束
   // 按降序排列
14    $sorted\_partition \leftarrow$ 
      $sort\_by\_adjusted\_utilization(sorted\_partition);$ 
15 结束
16 返回  $sorted\_partition;$ 
```

算法 2: 采用最佳适配的 $find_core_id$

输入: $sorted_partition, task$
输出: 处理器核索引或 $None$

```
1  $filtered\_core\_ids \leftarrow \{i \in$ 
    $\{0, \dots, |sorted\_partition| - 1\} :$ 
    $\bar{U}'_{sorted\_partition[i]} + \bar{U}'_{task} \leq 1\};$ 
2 若  $filtered\_core\_ids$  为  $\emptyset$  则
3   返回  $None;$ 
4 否则
5   // 返回第一个元素
6   返回  $filtered\_core\_ids[0];$ 
7 结束
```

品, 根据最佳适配、最差适配等策略寻找合适的箱子; BC 则逐个考虑箱子, 根据策略为当前箱子寻找合适物品。

以物品为中心的 **ICBFD** 伪代码见 Algorithms 1 与 2。在 Algorithm 1 的行 4 中, 函数 $find_core_id$ 返回按照最佳适配选中处理器核的索引, 或返回 $None$ 。 $None$ 表示无法按照最佳适配选择核的索引。Algorithm 1 中行 4 所用 $find_core_id$ 的伪代码, 见 Algorithm 2。Algorithm 2 采用降序最佳适配; 如果在 Algorithm 2 的行 5 中返回 $filtered_core_ids$ 的最后一个元素, 得到的就是降序最差适配算法。

以箱子为中心的 **BCBF** 伪代码见 Algorithms 3 与 4。

在 Algorithm 3 的行 5 中, 函数 $find_task$ 返回依据最

算法 3: 用 BCBF 寻找分区

输入: Γ
输出: Γ 的一个分区

```
1  $cur\_subset \leftarrow \emptyset;$   $partitioned\_tasks \leftarrow \emptyset;$ 
2  $free\_tasks \leftarrow \Gamma;$  // 尚未分配的任务
3 当  $free\_tasks \neq \emptyset$  执行
4   当  $True$  执行
5      $\tau \leftarrow find\_task(free\_tasks, cur\_subset);$ 
     // 未找到任务
6     若  $\tau = None$  则
7       将  $cur\_subset$  加入  $partitioned\_tasks;$ 
8        $cur\_subset \leftarrow \emptyset;$ 
9       continue;
10    结束
    // 找到任务
11    将  $\tau$  加入  $cur\_subset;$ 
    //  $cur\_subset$  中的所有任务均可调度
12    若  $is\_feasible(cur\_subset)$  则
13      从  $free\_tasks$  移除  $\tau;$ 
14      break;
15    结束
    //  $cur\_subset$  中并非所有任务都可调度
16    从  $cur\_subset$  移除  $\tau;$ 
17    将  $cur\_subset$  加入  $partitioned\_tasks;$ 
18     $cur\_subset \leftarrow \emptyset;$ 
19  结束
20 结束
21 将  $cur\_subset$  加入  $partitioned\_tasks;$ 
22 返回  $partitioned\_tasks;$ 
```

算法 4: 采用最佳适配的 $find_task$

输入: $free_tasks, cur_subset$
输出: 一个任务或 $None$

```
1  $filtered\_tasks \leftarrow \{\tau_i \in free\_tasks :$ 
    $\bar{U}'_{cur\_subset} + \bar{U}'_{\tau_i} \leq 1\};$ 
2 若  $filtered\_tasks = \emptyset$  则
3   返回  $None;$ 
4 结束
5 返回  $\tau_i \leftarrow \underset{\tau_i \in filtered\_tasks}{\operatorname{argmax}} \bar{U}'_{\tau_i};$ 
```

佳适配策略选中的任务, 或返回 $None$ 。 $None$ 表示无法按照最佳适配选中任务。Algorithm 3 中行 5 所用 $find_task$ 的伪代码, 见 Algorithm 4。Algorithm 4 采用最佳适配。如果将 Algorithm 4 第 5 行的“argmax”改为“argmin”, 得到的就是最差适配算法。

Algorithm 1 的行 9 以及 Algorithm 3 的行 12 中, 函数 $is_feasible$ 判断以参数给定的全部任务, 是否在 第 II 节所述意义下可调度。具体地, 设 cur_subset 为参数给出的任务集, DFP_i 为 $\tau_i \in cur_subset$ 的估计截止期违约概率。如果 $\forall \tau_i \in cur_subset, DFP_i \leq \rho_i$, 则任务集 cur_subset 可调度。 $is_feasible$ 所采用的具体方法在下一小节说明。

C. 可调度性分析

本小节说明所提方法采用的可调度性分析，具体是 $is_feasible$ 函数 (Algorithm 1 的行 9 以及 Algorithm 3 的行 12) 用于判断给定任务可调度性的分析。

可调度性分析采用已有方法 [17]。该方法计算 EDF 下最坏情况截止期违约概率 (WCDFP) 的上界。WCDFP 的定义由已有研究建立，以作业的截止期违约概率 (DFP) 为基础。

任务 τ_i 的 DFP 是 τ_i 所有作业的 DFP 中的最大概率。类似地，系统 DFP 是所有任务的 DFP 中的最大概率。WCDFP 则是任意作业到达序列下的最大 DFP。

所提方法使用的已有可调度性分析 [17]，对特定区间计算过载概率。区间 I 的过载概率，是释放时间及截止期均位于 I 内的作业，其执行时间之和超过区间长度 $|I|$ 的概率。可表示为下式。

$$\mathbb{P}(S_I > |I|) \quad (6)$$

I 为考察的区间； S_I 是服从分布 S_I 的随机变量，该分布由释放时间及截止期均位于 I 内的作业执行时间分布卷积得到。 S_I 计算如下：

$$S_I = \bigotimes_{\tau_i \in cur_subset} C_i \quad (7)$$

$N_i(I)$ 是从 τ_i 释放、且截止期位于 I 内的作业数。 cur_subset 是待分析任务集合，由 Algorithm 1 的行 9 和 Algorithm 3 的行 12 中 $is_feasible$ 的参数给出。

同一研究提出计算任务 WCDFP 上界的方法：将周期任务推广为偶发任务，并考虑使一个作业的 DFP 最大的最坏情况到达序列。

定理 1 (文献 [17] 的定理 1). 对于给定任务集 $part$ 中的任务 τ_k ，任务 DFP Φ_k 的上界为

$$\Phi_k^* = \sum_{t_s \in A(0, H - D_k)} \mathbb{P}(S_{[t_s, H]} > H - t_s), \quad (8)$$

其中 $A(0, H - D_k)$ 是 $[0, H - D_k]$ 内的全部到达时刻集合，此时所有任务都与绝对截止期 $d_{k, H/T_k} = H$ 的作业 $J_{k, H/T_k}$ 对齐。

使 $J_{i, j}$ 的 DFP 最大的最坏情况到达序列，是每个任务 τ_k 中某个作业的截止期与 $J_{i, j}$ 的截止期相同，而位于这些共同截止期作业之前的作业，以各自最小到达间隔周期性释放的序列。

如已有研究所述，由定理 1 得到的任务 WCDFP，与 cur_subset 中具体选择哪个任务无关，因为各任务的最坏情况到达序列相同。因此，由定理 1 得到的任务 WCDFP，就是 cur_subset 的 WCDFP。

已有研究 [17] 提出一种方法：用定理 1 分析 WCDFP 时，中途终止分析，以缩短分析时间。该方法的思路见文献 [17] 中的引理 12。

引理 1 (文献 [17] 的引理 12). 对于定理 1 中的到达序列 α^* 及给定区间 $[t_1, H]$ (其中 $t_1 \in A(0, H - D_k)$)，DFP Φ_k 中由满足 $t_s \in A(0, H - D_k)$ 和 $t_s < t_1$ 的区间 $[t_s, H]$ 贡献的部分 $\Phi_k^{t_s < t_1}$ ，具有以下上界

$$\Phi_k^{t_s < t_1} < \mathbb{P}_{busy}([t_1, H]), \quad (9)$$

其中 $\mathbb{P}_{busy}([t_1, H])$ 为处理器在区间 $[t_1, H]$ 内没有空闲的概率。

这种缩短分析时间的方法，可概括为：当前考察区间 $I = [t_x, H]$ 保持繁忙的概率 $P_{busy}(I)$ 下降到给定阈值以下时，停止计算。具体地，令 Φ_i^+ 表示计算 Eq. (8) 时遇到 t_x 的当前 Φ_i^* 估计值。一旦越过阈值，将 Φ_i^* 设为 $\Phi_i^+ + \mathbb{P}_{busy}(I)$ 。研究 [17] 的评估已表明，中途终止分析能显著缩短分析时间，因此也将该方法用于 Algorithm 1 的行 9 和 Algorithm 3 的行 12 中的可调度性分析。

D. 本文的过载概率计算方法

所提方法采用已有研究 [17] 未使用的方法，获得第 III-C 节中说明的 S_I ，最终计算过载概率。定理 1 和引理 1 使我们能够计算 WCDFP。为了按照引理 1 在中途截断分析，计算定理 1 中区间 $[t_s, H]$ 的过载概率时，按 t_s 降序进行。

设 t'_s 为计算完区间 $[t_s, H]$ 的过载概率后，下一个要考察过载概率的到达时刻。比较区间 $[t_s, H]$ 与 $[t'_s, H]$ 内的作业，唯一区别是在 t'_s 释放的作业。因此，计算 $S_{[t'_s, H]}$ 时，可按下式复用 $S_{[t_s, H]}$ ：

$$S_{[t'_s, H]} = S_{[t_s, H]} \otimes \bigotimes_{\tau_i \in Tasks(t'_s)} C_i \quad (10)$$

$Tasks(t'_s)$ 是在 t'_s 释放作业的全部任务集合。采用 Eq. (10) 后，计算 $S_{[t'_s, H]}$ 时不再需要将区间 $[t'_s, H]$ 内所有作业的执行时间分布重新卷积。

IV. 将所提方法应用于联邦调度

本节提出将第 III 节所提方法用于联邦调度的方式。该方法的优势是：所提方法能够进行概率分析，从而改善传统方法过高的核数需求，提高联邦调度的可调度性。联邦调度 [21] 是一种在多核处理器上高效调度并行实时任务的方法。它把任务集划分为重任务与轻任务；分析时为每个重任务分配专用核，再用重任务分析后剩余的核分析轻任务。将所提方法应用于联邦调度的流程如下。

- 1) 将满足 Eq. (2) 的 DAG 任务归类为轻任务，其余归类为重任务。
- 2) 使用任意合适的分析方法，计算重任务所需核数 m_{heavy} 。
- 3) 使用第 III 节所提方法，计算轻任务所需核数 m_{light} 。
- 4) 若 $m_{heavy} + m_{light}$ 不超过可用核数，则判定任务集可调度。

V. 评估

本节通过实验，评估所提任务集分区方法在 DAG 任务集总利用率变化时的可调度性。内容包括：用于比较的已有方法 (第 V-A 节)、DAG 任务集生成设置 (第 V-B 节)、与非概率分区的比较 (第 V-C 节)、与第 V-A 节所述已有方法的比较 (第 V-D 节)、不同启发式之间的比较 (第 V-E 节)、联邦调度的比较 (第 V-F 节)，以及研究发现的总结 (第 V-G 节)。

表 II: 任务集生成参数与评估系统配置

任务参数	说明
周期	从 1,000 到 10,000, 步长为 1,000
节点数	从二到五
源节点与汇节点数	各一个
入度与出度	从二到五
最坏情况利用率	第 V-D 节中从 0.5 到 1.0, 步长为 0.1; 节 V-C 与 V-E 中从 0.1 到 1.0
阈值 ρ	0.0001
任务集参数	说明
最坏情况利用率	从 0.8 到 8.0, 步长为 0.4
其他参数	说明
核数	四
每个利用率下的任务集数	1,000
每个任务集的分析超时	1,000 秒

A. 比较方法

在具有概率执行时间的 DAG 任务现有可调度性分析方法 [11]–[13] 中, 本节选择最具有可比性的一种。用于比较的已有方法 [11] 分三步计算 DAG 任务概率响应时间的上界: 局部响应时间 (LRT)、隔离情况下的响应时间 (RTI) 以及全局响应时间 (GRT)。从源节点到汇节点, 对每个节点迭代计算这些时间, 分别考虑前驱节点 (LRT)、同一 DAG 内的并行节点 (RTI) 以及更高优先级 DAG 任务 (GRT)。汇节点的 GRT 即代表 DAG 任务的响应时间。该方法 [11] 需要为每个 DAG 节点分配核和优先级, 但没有提出核分配方法。本文评估将其用于 Algorithm 1 的行 9 和 Algorithm 3 的行 12 中的 `is_feasible` 函数; 核分配由 Algorithm 1 确定, 同一 DAG 任务的节点分配到同一核。与所提方法的比较在第 V-D 节进行。

B. 评估设置

评估使用随机生成的 DAG 任务集, 单个 DAG 任务通过 RD-Gen [22] 的“扇入/扇出”功能生成, 并适配本文模型。源节点和汇节点数量固定为各一个, DAG 的入度和出度从一到三中随机选择。

每个任务的周期从 1,000 到 10,000, 步长为 1,000。每个 DAG 任务的节点数取二到五。第 V-D 节中每个 DAG 任务的最坏情况利用率从 0.5 到 1.0, 步长为 0.1; 节 V-C 与 V-E 中则从 0.1 到 1.0。通过随机选择任务, 直到达到目标任务集利用率, 生成任务集。

为了使生成的 DAG 适配本文模型, 根据给定 WCET 设置每个节点的概率执行时间。各可能执行时间的概率设为 $\mathbb{P}(WCET/3) = 0.98$ 和 $\mathbb{P}(WCET) = 0.02$, 反映 CARET 测量得到的 Autoware 运行时间经验分布, 文献 [23] 的评估设置也采用了这一分布。评估参数列于表 II。

每个 DAG 任务的阈值 ρ 设为 0.0001。假设系统有四个核。为评估大量任务集并提高评估可靠性, 将每个任务集的分析超时设为 1,000 秒。

评估中, 所有方法均以 Python 实现, 并在配备 5.3 GHz、24 核 AMD Ryzen Threadripper 7960X CPU、256 GiB RAM 及 Ubuntu 22.04 LTS (64 位) 操作系统的机器上执行。

C. 与基于 WCET 的方法比较

在分区时不考虑概率执行时间的 ICWFD, 与所提方法的可调度任务集数量比较, 见图 2。

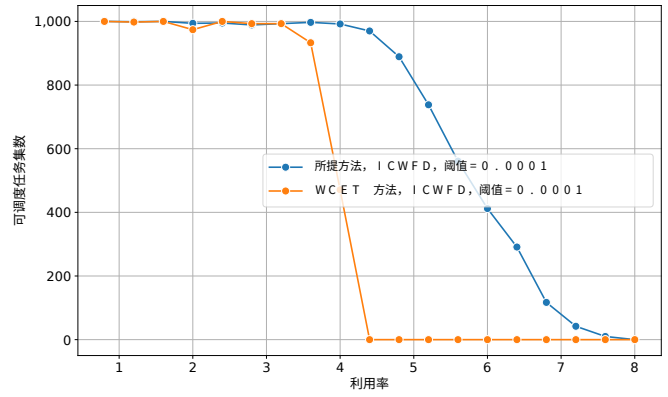


图 2: 所提方法与不考虑概率执行时间的方法, 在可调度任务集数量上的比较。

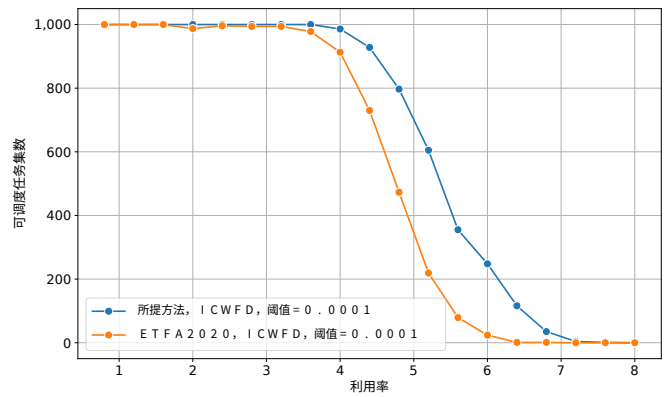


图 3: 所提方法与已有方法在可调度任务集数量上的比较。

“WCET 方法”基于最坏情况执行时间, 不考虑概率执行时间; 当任务集利用率超过四时, 其可调度任务集数量为零, 因为无论怎样配置任务集, 利用率超过四的任务集都无法装入四个核。

利用率低于四时, 考虑概率执行时间的“所提方法”能够调度几乎全部任务集; 但只考虑最坏情况执行时间的方法, 在利用率为 3.6 和 4.0 时无法调度部分任务集。推测原因是: 不考虑概率执行时间的 ICWFD, 即使核仍有余量, 有时也不允许选择任务分配给该核。利用率为 2.0 时, 可调度任务集数量未达到 1,000, 是因为部分任务集因分析超时而被判为不可调度。

D. 与已有方法比较

与第 V-A 节所述已有方法的评估结果, 见图 3 与 4。图例中的“所提方法”是第 III 节介绍的所提方法, “ETFA2020”是第 V-A 节解释的已有方法。图 3 比较采用 ICWFD 分区时的可调度任务集数量。由于分析时间随任务集中的任务数增长, 每个任务可能的利用率限制在 0.5 到 1.0。

图 3 表明, 所提方法“所提方法”能够调度与已有方法“ETFA2020”相同或更多的任务集。特别是在利用率为 5.2 时, 所提方法比已有方法多调度 38.6% (386 个任务集)。分析时间超过 1,000 秒的任务集被视为不可调度; 其中真正被判为不可调度的任务集比例至多为 1.3% (13 个任务集), 出现在利用率为 2.0 的“ETFA2020”中。

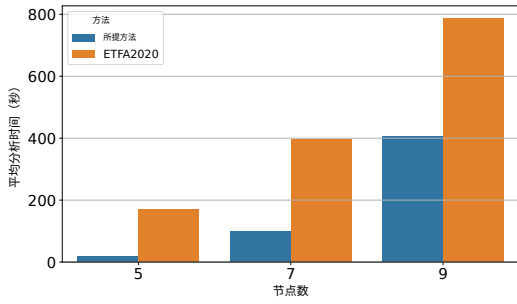


图 4: 每个 DAG 节点数变化时，所提方法与已有方法平均分析时间的比较。

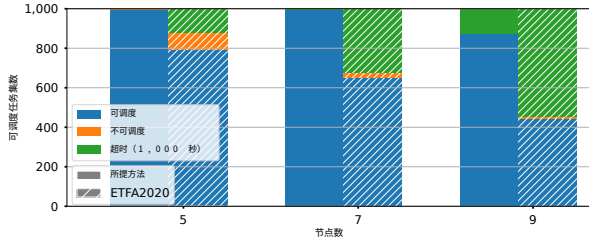


图 5: 所提方法与已有方法的任务集分类比较。

每个 DAG 节点数变化时，各任务集平均分析时间的比较见图 4；相同任务集按照可调度性分类后的结果见图 5。虽然每个超时任务集的分析时间超过 1,000 秒，但计算平均值时将其视为 1,000 秒。结果表明，每个 DAG 节点数变化时，所提方法的平均分析时间更短。随着节点数增加，“所提方法”的平均分析时间与“ETFA2020”的平均分析时间之比变小，而平均分析时间的差异似乎也变小。不过，查看图 5 中评估任务集的分类，“ETFA2020”在节点数为七时，超时比例超过 25%；节点数为九时，超过 50%。这意味着，不设置超时的平均分析时间，会高于本文评估所采用超时配置下的平均分析时间。

E. 所提方法采用不同启发式的比较

对第 III-B 节所述四种启发式 **ICBFD**、**ICWFD**、**BCBF** 和 **BCWF**，按可调度任务集数量进行评估，结果见图 6。随着任务集利用率变化，除 **ICBFD** 外，其余三种方法的可调度任务集数量比较接近。**ICBFD** 的数量低于其他方法，推测是因为它向缺少剩余容量的核分配任务。

ICBFD 将任务分配给利用率不超过一、且利用率最高的核。因此，可调度性分析时，该核上的任务集容易变为不可调度。此外，以物品为中心的算法在搜索核时，也会纳入此前已判为不可分配、且已无分配余量的核。

例如，两个核的利用率分别为 0.7 和 0.3，**ICBFD** 会为利用率为 0.2 的任务选择 0.7 的核，即使该核已勉强可调度。在考虑分配一个利用率为 0.2 的任务时，**ICBFD** 选择利用率为 0.7 的核，而该核已无力调度额外任务。由于待分配任务按利用率单调递减的顺序选择，算法会反复选择无余量的核，使可调度任务集数量低于其余三种方法。

其余三种方法按 **ICWFD**、**BCBF**、**BCWF** 的顺序，能够调度从多到少的任务集。**ICWFD** 搜索全部已启用的核，将任务分配到加入该任务后利用率最小的核；从保持可调度性角度看，这种策略似乎合理。**BCBF** 和 **BCWF** 是以

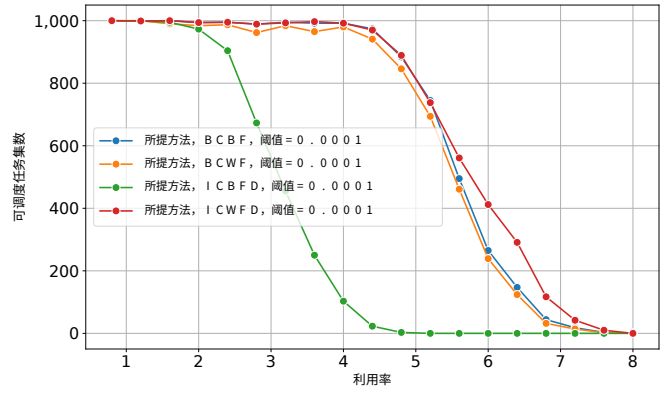


图 6: 不同启发式的可调度任务集数量比较。

箱子为中心的算法：依据策略，向当前核尽可能多地分配任务，再转到下一个核，重复同一过程。这两种方法的差异小于 5%。

F. 已有联邦调度与本文联邦调度的比较

本小节将第 IV 节提出的联邦调度，与一种已有的先进联邦调度方法 [24] 比较。本文在第 IV 节提出的联邦调度，对重任务采用文献 [24] 的重任务分析方法。

已有研究 [24] 明确界定 DAG 并行度，以推导 DAG 任务响应时间上界，并提出将其应用于联邦调度。该方法考虑节点执行时间，利用 DAG 链分解，提高了可调度性。该方法 [24] 将满足 $vol(G_i) > D_i$ 的 DAG 任务归类为重任务，其余归类为轻任务，并依据文献 [24] 的算法 2 计算重任务核数需求。 $vol(G_i)$ 是节点 WCET 的总和。轻任务采用已有分区 EDF [25] 调度。

评估假设环境有八个核；为生成重任务，每个 DAG 任务利用率从 0.1 到 2.0，步长为 0.1。任务集中的随机节点数范围为五到九，在 500 个 DAG 任务上，最坏情况利用率从 1.0 到 12.0。其他设置与表 II 相同。

第 IV 节所提方法与最新联邦方法 [24] 的评估结果，见图 7。横轴表示任务集利用率，纵轴表示任务集数量。“已有方法”是最新联邦方法 [24]，“已有+所提方法”是第 IV 节所提方法。图 7 中可调度任务集的比例表明，从任务利用率为 3.5 开始，所提方法能够调度更多任务集。由于假设八核环境，横轴上任务集利用率超过 8.0 的任何任务集，都不能被不考虑概率执行时间的已有方法调度。另一方面，所提方法考虑概率执行时间，因此能够调度部分利用率超过核数的任务集。在利用率为 3.0 和 3.5 时，所提方法将部分任务集判为超时，原因是轻任务可调度性分析——即第 III-C 节所提方法——比较耗时。

G. 研究启示

通过与最坏情况利用率的比较，验证了调整后平均利用率的有效性。采用考虑任务执行时间发生概率的调整后平均利用率后，图 2 表明：即使任务集最坏情况利用率超过用于调度的核数，仍然可以完成调度。

通过与已有 DAG 任务集可调度性分析方法比较，验证了所提方法更高的可调度性，以及从平均分析时间角度看，对更复杂任务集分析的适用性。两种方法的可调度性能比较中，图 3 显示所提方法能够调度更多任务集。增加每个 DAG 任务的节点数后，其平均分析时间仍低于已有方法，

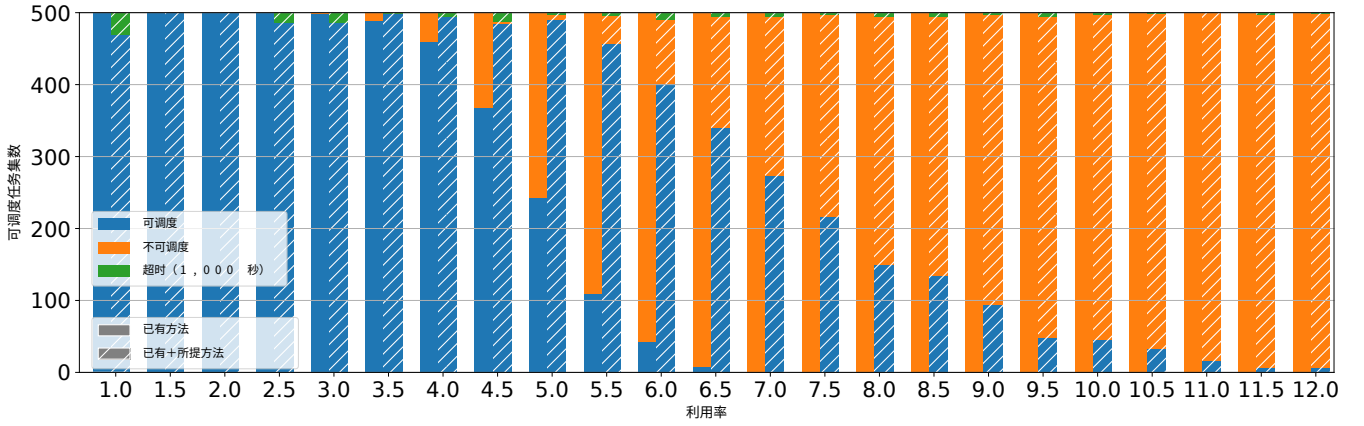


图 7: 本文联邦调度与先进联邦调度方法的任务集分类比较。

表 III: 所提方法与相关工作的比较

方法	EDF ^{a)}	SA ^{b)}	DAG ^{c)}	MC ^{d)}	NDC ^{e)}
RTSS 2021 [17]	✓	✓			
IEEE TC 2022 [13]	✓	✓	✓	✓	
JSA 2024 [12]	✓	✓	✓	✓	
ETFA 2020 [11]		✓	✓	✓	✓
RTSS 2023 [26]	✓		✓	✓	✓
所提方法	✓	✓	✓	✓	✓

a) EDF: 系统模型采用 EDF 策略

b) SA: 能够进行可调度性分析

c) DAG: 任务模型采用 DAG

d) MC: 系统模型假设多核环境

e) NDC: 不同 DAG 的节点可以调度到同一核

图 4 与 5 显示所提方法甚至能够分析这样的复杂任务集。需要注意，图 4 中任何在 1,000 秒处超时的任务集，都计为分析时间恰好 1,000 秒，这可能使报告的平均分析时间向下偏移。

根据对四种启发式 **ICBFD**、**ICWFD**、**BCBF** 和 **BCWF** 的比较，在可调度性方面，**ICWFD** 被确定为所提方法最适合采用的启发式。比较四种方法的可调度性，图 6 显示可调度任务集数量按 **ICWFD**、**BCBF**、**BCWF**、**ICBFD** 的顺序递减。

关于应采用哪种方法，结论是：**ICWFD** 在可调度任务集比例方面最优，优于其余三种方法。

通过采用所提方法，验证了对无法在单核调度的 DAG 任务，可调度性分析性能得到改善。第 III 节所提方法受到第 II 节中**假设**的限制，只适用于某一类 DAG 任务。然而，按照第 IV 节所提方式，将该方法用于联邦调度后，就可以处理因**假设**而无法直接处理的任务；图 7 表明，所提方法能比已有联邦调度方法 [24] 调度更多任务集。

VI. 相关工作

本节回顾任务分区、概率可调度性分析和联邦调度方面的相关工作。

A. 分区

多核调度 [27] 包括全局调度（允许任务迁移）与分区调度（将任务分配给指定核）。分区通常采用基于启发式的装箱算法分配处理器核。这方面的研究包括同时考虑 CPU

频率与利用率的节能方法 [28]，以及利用任务周期进行速率单调调度的方法 [29]。

研究 [19] 提出基于加速因子的指标，用于评估核分配启发式，并认为其优于基于利用率界的指标。该指标表明，有效的分区应在较低加速因子下取得较高处理器利用率，从而接近最优。合理分配降序（RAD）算法按利用率非递增顺序排列任务，在这一指标下被认为是最优的；本文所提方法采用的降序最差适配，也属于此类算法。

B. 概率可调度性分析

概率可调度性分析需要处理由概率执行时间带来的多个状态，因此对大型任务集可能难以处理。重采样与统计分析 [15] 可以缓解这一问题。重采样通过减少“执行时间—概率”对的数量，缩小状态空间。统计分析则利用任务执行时间的均值、方差等统计特征，估计超过截止期的概率，避免直接卷积。

为降低卷积的计算复杂度，文献 [30] 提出使用 Fourier 变换进行循环卷积。传统线性卷积的计算复杂度为 $O(n^2)$ ，处理大规模概率分布时会成为瓶颈。相比之下，循环卷积使用 Fourier 变换，可以按 $O(n \log n)$ 的复杂度高效计算卷积。

单处理器固定优先级调度的响应时间分析，使用临界时刻概念 [31]。然而，该研究指出，直接将确定性的临界时刻定理推广到概率领域，不一定能够代表最坏情况。该研究也找出了适用于概率执行时间的临界时刻。

在单处理器 EDF 调度下，已有研究提出可调度性分析方法 [17]，本文将其纳入所提方法。该分析考虑最可能使某个作业违约的作业到达序列，并计算该作业的截止期违约概率，从而保证可调度性。

与 DAG 任务丰富的确定性可调度性分析研究 [27] 相比，DAG 任务的概率分析较少。除可调度性分析外，还有若干研究关注节点以概率执行时间描述的 DAG 任务模型 [26]，[32]。文献 [26] 提出在整个 DAG 上管理服务器预算的方法，以及基于服务器调度时分析 DAG 任务终止概率的方法，目标是减少 WCET 超限与终止率。该预算管理改善了终止概率。文献 [32] 通过限制连续 DAG 实例之间的并行执行假设，进一步改进文献 [26] 的预算管理策略。

据我们所知, 具有概率执行时间的 DAG 任务可调度性分析, 共有三项研究 [11]–[13]。研究 [13] 将自动驾驶系统建模为 DAG, 分析端到端延迟。其方法 [13] 按节点周期性将 DAG 划分为子图, 分析子图响应时间分布, 再推导总体延迟分布。由于该方法要求每个子图至少分配一个核, 不同子图中的节点不能共享核, 因此无法在第 V 节中与本文方法进行比较。研究 [12] 针对存在不确定性的实时应用, 提出使用 SHPC-DAG 模型与 LLP 算法的调度方法, 在概率执行时间和条件节点下优化安全—性能指标。在第 V-D 节中与本文比较的研究 [11], 分析具有概率执行时间的 DAG 任务。它假设每个 DAG 节点的固定优先级及核分配已给定, 提出响应时间分析方法, 通过迭代方程对执行时间分布进行卷积, 推导响应时间。

C. 联邦调度

联邦调度是一种用于并行实时任务的方法。近期研究涵盖混合关键性 [33]、任意截止期 DAG [34] 以及条件 DAG [24] 的联邦调度。对于概率情形, 已有一种面向概率执行时间 DAG 任务的联邦调度方法 [35]。该方法根据概率执行时间的期望, 将任务划分为重任务或轻任务, 保证期望延迟有界; 这一延迟量化任务超过截止期的程度。

VII. 结论

本文提出一种分区方法, 保证基于 pWCET 的 DAG 任务在分区 EDF 下可调度。该方法根据调整后平均利用率划分任务, 并在核分配过程中进行可调度性分析。将 DAG 任务视为单节点任务, 使先进的单处理器 EDF 可调度性分析 [17] 能够适用。通过与基于 WCET 的分区方法比较, 验证了调整后平均利用率的有效性。实验评估表明, 所提方法在可调度任务集数量及单个任务集平均分析时间方面, 优于已有 DAG 可调度性分析方法 [11]。此外, 比较四种启发式 (ICBFD、ICWFD、BCBF 和 BCWFD) 发现, ICWFD 的可调度任务集比例最高。将所提方法用于联邦调度后, 对于单核无法处理的高负载 DAG 任务集, 显示出优于已有方法 [24] 的调度效率。

未来工作应优先研究利用 DAG 任务并行性的可调度性分析方法。虽然本文通过单处理器分析提高效率, 但仍局限于可处理的 DAG 任务。计算量较大的 DAG 任务需要并行执行。因此, 希望开发利用并行性、最小化计算时间的高性能方法, 以扩大适用范围。

参考文献

- [1] “Waymo,” <https://waymo.com/>.
- [2] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monroy, T. Ando, Y. Fujii, and T. Azumi, “Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems,” in *Proc. of ACM/IEEE ICCPS*, 2018, pp. 287–296.
- [3] Q. He, J. Sun, N. Guan, M. Lv, and Z. Sun, “Real-Time Scheduling of Conditional DAG Tasks With Intra-Task Priority Assignment,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 42, no. 10, pp. 3196–3209, 2023.
- [4] Z. Houssam-Eddine, N. Capodiceci, R. Cavicchioli, G. Lipari, and M. Bertogna, “The HPC-DAG Task Model for Heterogeneous Real-Time Systems,” *IEEE Trans. Comput.*, vol. 70, no. 10, pp. 1747–1761, 2021.
- [5] A. Melani, M. Bertogna, V. Bonifaci, A. Marchetti-Spaccamela, and G. C. Buttazzo, “Response-Time Analysis of Conditional DAG Tasks in Multiprocessor Systems,” in *Proc. of ECRTS*, 2015, pp. 211–221.
- [6] Q. He, N. Guan, and M. Lv, “Longer Is Shorter: Making Long Paths to Improve the Worst-Case Response Time of DAG Tasks,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 43, no. 12, pp. 4519–4531, 2024.
- [7] S. Bozhko, F. Marković, G. von der Brüggen, and B. B. Brandenburg, “What Really is pWCET? A Rigorous Axiomatic Proposal,” in *Proc. of IEEE RTSS*, 2023, pp. 13–26.
- [8] L. BEKDEMİR and C. F. BAZLAMAÇCI, “Hybrid Probabilistic Timing Analysis,” in *Proc. of UYMS*, 2021, pp. 1–6.
- [9] R. I. Davis and L. Cucu-Grosjean, “A Survey of Probabilistic Timing Analysis Techniques for Real-Time Systems,” *LITES*, pp. 1–60, 2019.
- [10] J. de Barros Vasconcelos and G. Lima, “Possible Risks with EVT-based Timing Analysis: an Experimental Study on a Multi-core Platform,” in *Proc. of SBESC*, 2022, pp. 1–8.
- [11] S. Ben-amor, L. Cucu-Grosjean, M. Mezouak, and Y. Sorel, “Probabilistic Schedulability Analysis for Precedence Constrained Tasks on Partitioned Multi-core,” in *Proc. of IEEE ETFA*, 2020, pp. 345–352.
- [12] X. Deng, A. H. Sifat, S.-Y. Huang, S. Wang, J.-B. Huang, C. Jung, R. Williams, and H. Zeng, “Partitioned Scheduling with Safety-Performance Trade-Offs in Stochastic Conditional DAG Models,” *J. Syst. Archit.*, vol. 153, p. 103189, 2024.
- [13] H. Lee, Y. Choi, T. Han, and K. Kim, “Probabilistically Guaranteeing End-to-End Latencies in Autonomous Vehicle Computing Systems,” *IEEE Trans. Comput.*, vol. 71, no. 12, pp. 3361–3374, 2022.
- [14] S. Bozhko, G. von der Brüggen, and B. B. Brandenburg, “Monte Carlo Response-Time Analysis,” in *Proc. of IEEE RTSS*, 2021, pp. 342–355.
- [15] R. I. Davis and L. Cucu-Grosjean, “A Survey of Probabilistic Schedulability Analysis Techniques for Real-Time Systems,” *LITES*, vol. 6, no. 1, pp. 04:1–04:53, 2019.
- [16] F. Marković, T. Nolte, and A. V. Papadopoulos, “Analytical Approximations in Probabilistic Analysis of Real-Time Systems,” in *Proc. of IEEE RTSS*, 2022, pp. 158–171.
- [17] G. von der Brüggen, N. Piatkowski, K.-H. Chen, J.-J. Chen, K. Morik, and B. B. Brandenburg, “Efficiently Approximating the Worst-Case Deadline Failure Probability Under EDF,” in *Proc. of IEEE RTSS*, 2021, pp. 214–226.
- [18] K. Zagalo, Y. Abdeddaïm, A. Bar-Hen, and L. Cucu-Grosjean, “Response Time Stochastic Analysis for Fixed-Priority Stable Real-Time Systems,” *IEEE Trans. Comput.*, vol. 72, no. 1, pp. 3–14, 2023.
- [19] S. K. Baruah, “Partitioned EDF scheduling: a closer look,” *Real-Time Syst.*, vol. 49, pp. 715–729, 2013.
- [20] C. Mommessin, T. Erlebach, and N. V. Shakhlevich, “Classification and Evaluation of the Algorithms for Vector Bin Packing,” *Comput. Oper. Res.*, vol. 173, p. 106860, 2025.
- [21] J. Li, J. J. Chen, K. Agrawal, C. Lu, C. Gill, and A. Saifullah, “Analysis of Federated and Global Scheduling for Parallel Real-Time Tasks,” in *Proc. of ECRTS*, 2014, pp. 85–96.
- [22] A. Yano and T. Azumi, “RD-Gen: Random DAG Generator Considering Multi-rate Applications for Reproducible Scheduling Evaluation,” in *Proc. of IEEE ISORC*, 2023.
- [23] H. Toba and T. Azumi, “Deadline Miss Early Detection Method for DAG Tasks Considering Variable Execution Time,” in *Proc. of ECRTS*, Dagstuhl, Germany, 2024, pp. 8:1–8:21.
- [24] Q. He, N. Guan, Z. Jiang, and M. Lv, “On the degree of parallelism for parallel real-time tasks,” *J. Syst. Archit.*, vol. 156, p. 103286, 2024.
- [25] S. Baruah, “The federated scheduling of systems of conditional sporadic DAG tasks,” in *Proc. of EMSOFT*, 2015, pp. 1–10.
- [26] Z. Tong, S. Ahmed, and J. H. Anderson, “Holistically Budgeting Processing Graphs,” in *Proc. of IEEE RTSS*, 2023, pp. 27–39.
- [27] M. Verucchi, I. S. n. Olmedo, and M. Bertogna, “A Survey on Real-Time DAG Scheduling, Revisiting the Global-Partitioned Infinity War,” *Real-Time Syst.*, vol. 59, no. 3, p. 479–530, 2023.
- [28] A. Guasque, P. Balbastre, A. Crespo, and J. Coronel, “Energy Efficient Partition Allocation in Partitioned Systems,” *IFAC-PapersOnLine*, vol. 51, no. 10, pp. 82–87, 2018.
- [29] T. Wang, S. Homsy, L. Niu, S. Ren, O. Bai, G. Quan, and M. Qiu, “Harmonicity-Aware Task Partitioning for Fixed Priority Scheduling of Probabilistic Real-Time Tasks on Multi-Core

Platforms,” *ACM Trans. Embed. Comput. Syst.*, vol. 16, no. 4, 2017.

- [30] F. Marković, A. V. Papadopoulos, and T. Nolte, “On the Convolution Efficiency for Probabilistic Analysis of Real-Time Systems,” in *Proc. of ECRTS*, 2021, pp. 16:1–16:22.
- [31] K.-H. Chen, M. Günzel, G. von der Brüggen, and J.-J. Chen, “Critical Instant for Probabilistic Timing Guarantees: Refuted and Revisited,” in *Proc. of IEEE RTSS*, 2022, pp. 145–157.
- [32] Z. Tong and J. H. Anderson, “Budgeting Processing Graphs Under Restricted Parallelism,” in *Proc. of IEEE SIES*, 2024, pp. 172–182.
- [33] F. Guan, J. Lee, C. J. Xue, J.-M. Wu, and N. Guan, “Mixed-Criticality Federated Scheduling for Relaxed-Deadline DAG Tasks,” in *Proc. of IEEE RTSS*, 2024, pp. 362–374.
- [34] F. Guan, L. Peng, and J. Qiao, “A New Federated Scheduling Algorithm for Arbitrary-Deadline DAG Tasks,” *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2264–2277, 2023.
- [35] J. Li, K. Agrawal, C. Gill, and C. Lu, “Federated scheduling for stochastic parallel real-time tasks,” in *Proc. of IEEE RTCSA*, 2014, pp. 1–10.